

DataVortex: Social Engine Data Recovery & Intelligence Engine

A human-first, step-by-step breakdown of how corrupted social media intake streams were restored, how missing interactions were reconstructed using machine learning, and what actually drives virality and engagement across the platform.

The Story Behind the Project

Imagine waking up to an alerting dashboard: **the intake gateway for a social media platform crashed.**

How We Found & Diagnosed the Dataset in the Terminal

We jumped straight into the command line to inspect the aftermath:

```
# 1. Inspect what arrived in the crash directory
dir / ls

# 2. Peek into the corrupted tables using quick Python one-liners
python -c "import pandas as pd; df = pd.read_csv('Social_Engine_Posts_Corrupted.csv'); pr
```

When we loaded the raw crash dumps, the database was in total disarray. Post IDs were duplicated, like counts were negative, dates were recorded in three conflicting international standards, foreign characters were garbled, and thousands of records were missing their interaction metrics entirely.

How We Debunked the Common Assumptions

Before rushing into any fixes, we used data forensics in the terminal to debunk three big surface-level assumptions:

1. Debunked: "Negative likes mean massive unlikes or downvotes."

The Assumption: Did 525 posts trigger intense community backlash and get downvoted to -4,812?

The Reality: We examined the absolute values $|x|$ of those negative numbers. Their average was **\$2,457\$** and standard deviation was **\$1,426\$**—a carbon copy of normal positive likes (\$0 to 5000\$). Nobody got downvoted; an intake serialization glitch or signed 16-bit integer conversion simply inverted the sign bit. Applying `abs()` restored the genuine likes without guessing.

2. Debunked: "Missing likes mean zero likes."

The Assumption: If `likes` is blank/null, can we just fill it with `0`?

The Reality: Filling NaN with `0` created a silent mathematical disaster: it forced `like_ratio =`

0.0 even though posts had hundreds of shares and comments, and artificially shrunk `total_engagement`. It was an intake network loss, not zero engagement. We rebuilt the missing values using **CatBoost Regressor paired with Empirical Residual Sampling (Predictive Mean Matching)** to preserve the true uniform variance ($[0, 5000]$).

3. Debunked: "More followers guarantee more reach and engagement."

The Assumption: High-follower accounts (\$40,000+) drive the platform's conversation.

The Reality: Plotting followers against total engagement produced a flat scatter cloud with $r = -0.0109$. The platform does not rely on a static follower feed; an algorithmic recommendation engine treats micro-creators (<1k followers) equally, regularly awarding them 4,000+ interactions per post.

This project, **DataVortex**, documents the complete recovery operation:

1. **Triage & Forensic Cleaning:** Fixing corrupted records without fabricating data.
2. **Machine Learning Reconstruction:** Solving missing likes and ratio distortion using **CatBoost Regressor + Residual Sampling (PMM)**.
3. **Exploratory Data Analysis (EDA):** Uncovering counterintuitive truths about followers, algorithms, platform behaviors, and community sentiment.

Step 1: Cleaning the Corrupted Data (The Forensic Fixes)

We started with two raw, broken files:

- * `Social_Engine_Users.csv` (1,500 registered creator profiles)
- * `Social_Engine_Posts_Corrupted.csv` (12,360 corrupted post entries)

Here is what went wrong and how each issue was systematically resolved:

1. 360 Duplicate Rows (Server Retries)

- **What broke:** When the intake servers timed out, client apps kept resending the same payload. This wrote 360 identical records into the database.
- **The fix:** Filtered out exact duplicate rows based on unique `post_id`, restoring the post count to exactly **12,000 unique records**.

2. 525 Inverted (Negative) Likes

- **What broke:** 525 posts had negative numbers like `-4812.0` or `-1795.0`.
- **Why it happened:** A sign-bit inversion during serialization or a delta-stream bug in the message broker. The absolute values ($|x|$) had an average of \$2,457 and standard deviation of \$1,426—the exact same uniform spread as the positive likes ($[0, 5000]$).
- **The fix:** Corrected signs using `abs(round(v))`, recovering every single interaction with 100% fidelity.

3.3 Conflicting Date Formats

- **What broke:** Timestamps arrived in three incompatible standards:
- Unix Epoch seconds (e.g., `1722528840`)
- ISO 8601 strings (e.g., `2024-10-15T14:30:00`)
- European date strings (e.g., `25-09-2024`)
- **The fix:** Built a regex-driven multi-format parser that converted every single timestamp into **standardized UTC timestamps** (`timestamp_utc`).

4. Garbled Characters & The Singapore Edge Case

- **What broke:** HTML entities and corrupted multi-byte characters (`&` instead of `&` , `Ã©` instead of `é`).
- **The fix:** Cleaned strings using HTML unescaping and text encoding fixes. For location profiles where the city was `"Singapore"` but country was missing, we handled the city-state edge case by mapping both to `"Singapore"` .

Step 2: The Missing Likes & CatBoost Imputation

The Silent Bug

In the raw data, **1,814 posts (~15.1%)** were missing their `likes` count completely.

In initial processing pipelines, missing likes were filled with `0` to calculate engagement ratios. This created a **silent mathematical bug**:

- * Any row with missing likes ended up with `like_ratio = 0.0` .
- * `total_engagement` was deflated (only counting shares + comments).
- * Share and comment ratios were artificially inflated.

Why Simple Supervised Regression Failed (The Variance Collapse)

We initially evaluated training a standard supervised model (Linear, Ridge, or Random Forest) to predict likes from shares, comments, followers, and platforms.

However, an empirical test revealed:

- * Correlation between likes and shares: **$\$-0.0012\$$**
- * Correlation between likes and comments: **$\$+0.0100\$$**
- * Regression R^2 : **$\$\text{approx } 0.0006\$$** (almost zero mutual information).

Because regularized models like CatBoost recognize that features have near-zero correlation with likes, a standard regressor shrinks all predictions to the global mean ($\text{approx } 2,492$). **Imputing 1,814 rows with $\sim 2,492$ would collapse the variance and create an artificial spike right in the center of the distribution.**

The CatBoost + Residual Sampling Solution (PMM)

To preserve the true distribution and natural variance ($U(0, 5000)$), we implemented **CatBoost Regressor with Empirical Residual Sampling (Predictive Mean Matching)**:

1. Architecture:

- `cat_features`: `platform`, `language`, `country`, `post_day_of_week`
- `text_features`: `text_content` (tokenized natively by CatBoost)
- Numerical: `shares`, `comments`, `follower_count`, `post_hour`, `account_age_days`

2. Residual Sampling:

$$\hat{y}_{\text{imputed}} = \text{clip}(\hat{y}, 0, 5000) + \epsilon_{\text{sampling}}$$

3. Full Metric Recalculation:

- $\text{total_engagement} = \text{likes}_{\text{cleaned}} + \text{shares} + \text{comments}$
- $\text{like_ratio} = \frac{\text{likes}}{\text{total_engagement}}$
- Added an audit flag `likes_imputed = True` for transparency.

Imputation Results:

Metric	Non-Missing Likes (\$N=10,186\$)	CatBoost Imputed (\$N=1,814\$)	Combined Dataset (\$N=12,000\$)
Mean	\$2,491.86\$	\$2,447.38\$	\$2,485.02\$
Std Dev	\$1,437.83\$	\$1,402.15\$	\$1,431.09\$
Missing Likes	\$0\$	\$0\$	\$0\$
Mean Like Ratio	\$0.578\$	\$0.572\$	\$0.577\$

Step 3: Key Insights from Exploratory Data Analysis (EDA)

With the clean master dataset ([cleaned_dataset.csv](#)), we analyzed what actually drives performance on the Social Engine:

1. The Follower Myth (\$r = -0.0109\$)

- **Finding:** Follower count has **zero correlation** with post engagement. Accounts with 500 followers frequently pull 4,000+ interactions on a single post, achieving engagement rates exceeding **500%**.
- **Takeaway:** The platform does not rely on a traditional follower feed. It operates via recommendation algorithms (like TikTok or YouTube Shorts) where content stands on its own merits regardless of creator audience size.

2. The Universal Interaction Rule (50 / 33 / 17 Split)

- Across every platform, total engagement reliably divides into:
- **~50% Likes** (quick agreement / passive reaction)
- **~33% Shares** (amplification & viral reach)
- **~17% Comments** (deep discussion & friction)
- **Reddit** leads in comment depth (ideal for honest product feedback and unboxings).
- **Twitter** leads in share ratio (33.8%, making it the top channel for viral distribution).

3. The "Friction Effect" (Sentiment Dynamics)

- Posts expressing **mixed feelings or critical reviews generate higher comment counts** (averaging 523 comments vs. 505 for praise posts).
- Controversial, authentic, or problem-solving topics spark community debate. Brands that engage constructively in critical comment threads build higher brand loyalty.

4. Global Posting Cadence

- **Top Countries:** United States (13.7%), China, Germany, Brazil, and Japan.
 - **Cadence:** Consistent volume across all 7 days of the week (<6% weekend variance) and across 8 major languages (~1,500 posts each).
-

Repository Structure & Artifacts

```
datavortex/
├─ README.md # This file (Complete human-readable summary)
├─ EDA_Report.md # Full executive report with embedded figures &
├─ Social_Engine_Intake_Restoration.ipynb # Interactive Jupyter Notebook with data restor
├─ cleaned_dataset.csv # Final 12,000-row master dataset (0 nulls, all
├─ Social_Engine_Posts_Cleaned.csv # Cleaned post entity table
├─ Social_Engine_Users_Cleaned.csv # Cleaned user entity table
├─ Social_Engine_Posts_Corrupted.csv # Original raw corrupted posts (preserved untou
├─ Social_Engine_Users.csv # Original raw users table (preserved untouched
├─ figures/ # Exported charts & visualizations
│   └─ metrics_distribution.png
│   └─ followers_vs_engagement.png
│   └─ platform_engagement_comparison.png
│   └─ engagement_composition_ratios.png
│   └─ brand_performance_benchmark.png
│   └─ sentiment_engagement_dynamics.png
│   └─ temporal_trends.png
│   └─ geo_language_distribution.png
└─ catboost_info/ # Training logs and telemetry from CatBoost
```

How to Run & Reproduce

1. Prerequisites

Install the required dependencies:

```
pip install pandas numpy scikit-learn catboost matplotlib seaborn jupyter
```

2. Inspect or Run the Notebook

Open the interactive Jupyter Notebook to step through data extraction, cleaning, and visualizations:

```
jupyter notebook Social_Engine_Intake_Restoration.ipynb
```

3. Quick Load in Python

```
import pandas as pd

# Load master cleaned dataset
df = pd.read_csv("cleaned_dataset.csv")

print(f"Total Posts: {len(df):,}")
print(f"Missing Likes: {df['likes_cleaned'].isnull().sum()}")
print(f"Average Engagement: {df['total_engagement'].mean():.1f}")
```



Summary Checklist

- [x] Preserved raw corrupted files untouched for reproducible audit trails.
- [x] Eliminated 360 retry duplicates and corrected 525 sign-inverted likes.
- [x] Standardized 3 mixed timestamp formats into ISO UTC.
- [x] Solved 1,814 missing likes using CatBoost + Residual Sampling (PMM), preventing variance collapse.
- [x] Recalculated all derived interaction ratios with 100% mathematical integrity.
- [x] Validated findings with clear executive documentation and visual figures.

ThankYa fella!!!!!!!!!!!!